PEP 518 – Specifying Minimum Build System Requirements for Python Projects

Abstract

This PEP specifies how Python software packages should specify what build dependencies they have in order to execute their chosen build system. As part of this specification, a new configuration file is introduced for software packages to use to specify their build dependencies (with the expectation that the same configuration file will be used for future configuration details).

Rationale

When Python first developed its tooling for building distributions of software for projects, distutils [1] was the chosen solution. As time went on, setuptools [2] gained popularity to add some features on top of distutils. Both used the concept of a setup.py file that project maintainers executed to build distributions of their software (as well as users to install said distribution).

Using an executable file to specify build requirements under distutils isn't an issue as distutils is part of Python's standard library. Having the build tool as part of Python means that a setup.py has no external dependency that a project maintainer needs to worry about to build a distribution of their project. There was no need to specify any dependency information as the only dependency is Python.

But when a project chooses to use setuptools, the use of an executable file like setup.py becomes an issue. You can't execute a setup.py file without knowing its dependencies, but currently there is no standard way to know what those dependencies are in an automated fashion without executing the setup.py file where that information is stored. It's a catch-22 of a file not being runnable without knowing its own contents which can't be known programmatically unless you run the file.

Setuptools tried to solve this with a setup_requires argument to its setup() function [3]. This solution has a number of issues, such as:

- No tooling (besides setuptools itself) can access this information without executing the setup.py, but setup.py can't be executed without having these items installed.
- While setuptools itself will install anything listed in this, they won't be installed until during the execution of the setup() function, which means that the only way to actually use anything added here is through increasingly complex machinations that delay the

import and usage of these modules until later on in the execution of the setup() function.
- This cannot include setuptools itself nor can it include a replacement to setuptools, which means that projects such as numpy.distutils are largely incapable of utilizing it and projects cannot take advantage of newer setuptools features until their users naturally upgrade the version of setuptools to a newer one.
- The items listed in setup_requires get implicitly installed whenever you execute the setup.py but one of the common ways that the setup.py is executed is via another tool, such as pip, who is already managing dependencies. This means that a command like pip install spam might end up having both pip and setuptools downloading and installing packages and end users needing to configure both tools (and for setuptools without being in control of the invocation) to change settings like which repository it installs from. It also means that users need to be aware of the discovery rules for both tools, as one may support different package formats or determine the latest version differently.

This has culminated in a situation where use of setup_requires is rare, where projects tend to either simply copy and paste snippets between setup.py files or they eschew it all together in favor of simply documenting elsewhere what they expect the user to have manually installed prior to attempting to build or install their project.

All of this has led pip [4] to simply assume that setuptools is necessary when executing a setup.py file. The problem with this, though, is it doesn't scale if another project began to gain traction in the community as setuptools has. It also prevents other projects from gaining traction due to the friction required to use it with a project when pip can't infer the fact that something other than setuptools is required.

This PEP attempts to rectify the situation by specifying a way to list the minimal dependencies of the build system of a project in a declarative fashion in a specific file. This allows a project to list what build dependencies it has to go from e.g. source checkout to wheel, while not falling into the catch-22 trap that a setup.py has where tooling can't infer what a project needs to build itself. Implementing this PEP will allow projects to specify what build system they depend on upfront so that tools like pip can make sure that they are installed in order to run the build system to build the project.

To provide more context and motivation for this PEP, think of the (rough) steps required to produce a built artifact for a project:

1. The source checkout of the project.
2. Installation of the build system.
3. Execute the build system.

This PEP covers step #2. PEP 517 covers step #3, including how to have the build system dynamically specify more dependencies that the build system requires to perform its job. The purpose of this PEP though, is to specify the minimal set of requirements for the build system to simply begin execution.

Specification

File Format

The build system dependencies will be stored in a file named pyproject.toml that is written in the TOML format [5].

This format was chosen as it is human-usable (unlike JSON [6]), it is flexible enough (unlike configparser [8]), stems from a standard (also unlike configparser [8]), and it is not overly complex (unlike YAML [7]). The TOML format is already in use by the Rust community as part of their Cargo package manager [12] and in private email stated they have been quite happy with their choice of TOML. A more thorough discussion as to why various alternatives were not chosen can be read in the Other file formats section. The authors do realize, though, that choice of configuration file format is ultimately subjective and a choice had to be made and the authors prefer TOML for this situation.

Below we list the tables that tools are expected to recognize/respect. Tables not specified in this PEP are reserved for future use by other PEPs.

build-system table

The [build-system] table is used to store build-related data. Initially only one key of the table will be valid and is mandatory for the table: requires. This key must have a value of a list of strings representing PEP 508 dependencies required to execute the build system (currently that means what dependencies are required to execute a setup.py file).

For the vast majority of Python projects that rely upon setuptools, the pyproject.toml file will be:

```
[build-system]
# Minimum requirements for the build system to execute.
requires = ["setuptools"]  # PEP 508 specifications.
```

Because the use of setuptools is so expansive in the community at the moment, build tools are expected to use the example configuration file above as their default semantics when a pyproject.toml file is not present.

Tools should not require the existence of the [build-system] table. A pyproject.toml file may be used to store configuration details other than build-related data and thus lack a [build-system] table legitimately. If the file exists but is lacking the [build-system] table then the default values as specified above should be used. If the table is specified but is missing required fields then the tool should consider it an error.

tool table

The [tool] table is where any tool related to your Python project, not just build tools, can have users specify configuration data as long as they use a sub-table within [tool], e.g. the flit tool would store its configuration in [tool.flit].

We need some mechanism to allocate names within the tool.* namespace, to make sure that different projects don't attempt to use the same sub-table and collide. Our rule is that a project can use the subtable tool.$NAME if, and only if, they own the entry for $NAME in the Cheeseshop/PyPI.

JSON Schema

To provide a type-specific representation of the resulting data from the TOML file for illustrative purposes only, the following JSON Schema [13] would match the data format:

```
{
    "$schema": "http://json-schema.org/schema#",

    "type": "object",
    "additionalProperties": false,

    "properties": {
        "build-system": {
            "type": "object",
            "additionalProperties": false,

            "properties": {
                "requires": {
                    "type": "array",
                    "items": {
                        "type": "string"
                    }
                }
            },
            "required": ["requires"]
        },

        "tool": {
            "type": "object"
        }
    }
}
```

Rejected Ideas

A semantic version key

For future-proofing the structure of the configuration file, a semantics-version key was initially proposed. Defaulting to 1, the idea was that if any semantics changes to previously defined keys or tables occurred which were not backwards-compatible, then the semantics-version would be incremented to a new number.

In the end, though, it was decided that this was a premature optimization. The expectation is that changes to what is pre-defined semantically in the configuration file will be rather conservative. And in the instances where a backwards-incompatible change would have occurred, different names can be used for the new semantics to avoid breaking older tools.

A more nested namespace

An earlier draft of this PEP had a top-level [package] table. The idea was to impose some scoping for a semantics versioning scheme (see A semantic version key for why that idea was rejected). With the need for scoping removed, the point of having a top-level table became superfluous.

Other table names

Another name proposed for the [build-system] table was [build]. The alternative name is shorter, but doesn't convey as much of the intention of what information is stored in the table. After a vote on the distutils-sig mailing list, the current name won out.